CAREERKIT

SIGNAL OUT OF NOISE

A job search engine you operate by talking to it. In one August 2026 reference run, it read 19,550 postings and surfaced 161.

CareerKit / open source, MIT
github.com/omoji-personal/careerkit

Finding job postings was never the hard part. The hard part is that almost everything you find is noise, and the few that matter are buried by the time you reach them. CareerKit polls employer applicant tracking systems directly, scores every posting against rules you wrote yourself, and hands you the short list with its reasoning attached.

It is built for people who do not write code. You talk to Claude Code; Claude runs the machinery.

SPECIFICATION

ATS platforms polled	20	Public job feeds	14 + 1 dormant
Auto-discoverable	14 probeable; 6 URL-only	Aug. 2026 reference run	19,550 read, 161 passed
Regression suite	500+ cases	Continuous integration	3.10 / 3.12 / 3.13 / 3.14
Engine	Python, SQLite	Licence	MIT

ONE

What it does

CareerKit watches the job boards employers actually post on, rather than the search pages that aggregate them. That distinction is most of the value: an employer's own applicant tracking system lists the requisition the day it opens, with its real title, location and requisition id, and it stops listing it the day it closes.

The 20 supported ATS families are platform capacity, not a promise of comprehensive sourcing. A run reaches only the active employer boards and operational feeds you have configured. Coverage is therefore reported as an auditable sourcing surface, with gaps, rather than as a claim that every possible opening was searched.

Every posting it collects is scored against your rules. The output is a report, ordered, with the reasoning attached to each entry, so you can tell the difference between a role that matched and a role that merely appeared.

The ratio is the product

In an August 2026 reference run against the 273 employer-board rows and 14 feeds configured at that time, CareerKit pulled 19,550 postings. 275 were in the right job family at all, and of those **161** cleared every screenable rail. Everything else was screened out against a named rule, and `audit` shows you those decisions whenever you want to argue with them.

> WHY THIS MATTERS
>
> A filter you cannot inspect is a filter you cannot trust. Screened-out postings are not kept in the database, so `audit` re-polls the boards and re-scores them to show you what was killed and why. That is the command that lets you overrule your own gates.

What it is not

It is not a mass-application tool. It fills a form and stops at submit for your review. You handle every certification; any agent click requires fresh approval for that one completed form. It does not create accounts, handle passwords, or answer captcha or OTP checks; you complete those directly and never relay codes.

TWO

Setup, and why the interview is not optional

You need Claude Code (Pro plan minimum), Python 3.10 or newer, and git. Install Claude Code with `npm install -g @anthropic-ai/claude-code`, which needs Node 18 or newer. Chrome and the Claude-in-Chrome extension are optional, and only needed if you want it filling application forms for you. macOS and Linux work as they are; on Windows, run the shell scripts from Git Bash or WSL.

```
git clone https://github.com/omoji-personal/careerkit.git
cd careerkit
./setup.sh          # checks prerequisites, builds .venv, installs deps
claude              # open Claude Code in this folder
/setup              # about 40 minutes
```

`/setup` interviews you: what you do, what you want next, where you will work, what you will not do, and what is true about you. It writes `profile/profile.yaml` and `profile/claims.md`.

It is not skippable. Until that file exists the engine has no rules to score against, and it will tell you so rather than guess. A tool that guesses your criteria produces a report that looks exactly as confident as a correct one.

The claims register

`profile/claims.md` is the list of things that may be said about you. Nothing goes into a resume, cover letter, form answer or outreach message unless it is in there. When you tell Claude something new about yourself, it gets added, with your confirmation, before it is used.

THREE

How scoring works: lanes, rails, gates

Lanes are the job families you want, each with its own titles and weight. **Rails** are the hard constraints: location, compensation floor, clearance, product specialisations you do not hold, role families you rule out. **Gates** are the four possible verdicts.

VERDICT	MEANING	WHAT TO DO
QUALIFIED	Passes every screenable rail, with evidence found in the posting.	Read it. This is the short list.
VERIFY	Looks right, but one rail could not be evidenced from the text.	Check the one open question yourself.
SLOT-BLOCKED	Clears the rails, but the role family is one you rule out.	Nothing, unless your criteria changed.
EXCLUDED	Fails a hard rail. Never surfaces in the report.	Review in bulk with `audit`.

Your rules are the only rules

`profile/profile.yaml` is the single place scoring rules exist. To change what surfaces, change that file, usually by asking Claude to run `/criteria`. Never by editing engine code.

This is enforced, not merely intended: a test parses every module in `engine/` and fails the build if one of a list of person-specific terms appears outside a docstring. It is a tripwire rather than a proof, but it means the engine cannot quietly acquire an opinion about what you want.

Two kinds of exclusion

`exclusions.titles` is what you would rather avoid, and an employer you marked as a dream waives it. `exclusions.titles_always` is what you cannot do, and nothing waives it. Wanting to work somewhere badly does not qualify you for a job there, and treating those two as the same thing fills a report with roles at exciting companies that you have no real chance at.

> IF A RULE IS UNUSABLE
>
> Malformed structure or invalid values stop with the exact field named. Unknown keys and unusable regexes in non-exclusion scoring terms are reported and skipped. An unusable exclusion stops the run: dropping it would fail open and silently surface everything it was meant to ban. Blank company-exclusion strings are filtered out.

FOUR

The daily loop

`/search`	Sweep every board and feed, report what is genuinely new since the last run.
`/evaluate <url>`	An honest fit read on any posting, from anywhere. Names the real disqualifiers.
`/apply <url>`	Build and fill; you review and approve this one submission.
`/prep <company>`	Interview preparation from your own story bank.
`/track`	Where everything stands.
`/ingest <url>`	Score and register anything you found elsewhere, so the engine knows about it.
`/compose`	Resume, cover letter and profile work, gated by your claims register.
`/outreach`	Referrals, thank-yous and nudges. Draft only by default.
`/criteria`	Change rules, then rescore stored postings and pull to recover jobs the old gates excluded.
`/audit`	Review what the gates killed. Run it monthly.

Running the engine directly

Claude does this for you, but every operation is also available from a CareerKit source checkout:

`./careerkit.py pull` poll, score, report	`./careerkit.py audit` sample exclusions by reason
`./careerkit.py discover` find employer boards	`./careerkit.py search` open-web ATS discovery
`./careerkit.py queries` print the query matrix	`./careerkit.py ingest-url` register one pasted URL
`./careerkit.py ingest-urls` register a URL file	`./careerkit.py verify` confirm board ownership
`./careerkit.py report` rebuild or export	`./careerkit.py analytics` conversion and follow-ups
`./careerkit.py progress` record a pipeline stage	`./careerkit.py relationship` employer contacts and context
`./careerkit.py status` database and source health	`./careerkit.py db` integrity check or backup
`./careerkit.py rescore` apply changed criteria	`./careerkit.py profile-lint` validate profile rules
`./careerkit.py coverage` unique endpoints and sourcing gaps	`./careerkit.py doctor` setup and drift check
`./careerkit.py tracker-sync` preview or repair tracker drift	`./careerkit.py enrich` refresh thin postings
`./careerkit.py applied` reconcile application evidence	`./careerkit.py consistency` compare report and database
`./careerkit.py voice-lint` flag machine-written patterns	`./careerkit.py claims-lint` check claims in a draft
`./careerkit.py history` show status events	`./careerkit.py mark` set a posting status

FIVE

Calibration: keeping the filter honest

A screening tool drifts toward the mistake you cannot see. Rails that are slightly too tight remove real matches, and because you never see what was removed, the reports keep looking reasonable. This is the failure that costs the most and announces itself the least.

`audit` is the answer. It re-scores, groups every exclusion reason, and shows representative postings from each. You argue with it, and what you disagree with becomes a profile change.

> RUN IT MONTHLY
>
> A filter nobody reviews is a filter nobody can trust. Thirty minutes of arguing with your own gates is worth more than any amount of additional sourcing.

Zero results is a claim

"Nothing new this week" can mean nothing was posted, or it can mean a board changed its API and has been returning an error for eleven days. CareerKit tracks the health of every source separately, reports consecutive failures, and flags a source that returned nothing when it had postings last run. A quiet week has to earn the description.

`coverage` distinguishes active employer rows from unique configured board endpoints and reports operational feeds separately: duplicate registry rows do not increase real reach, and a keyed feed without its required configuration is dormant. A partial response or a run stopped at a configured or provider cap is still a coverage gap even if it returned jobs; it must not be described as healthy or complete.

SIX

What leaves your machine

Worth being precise about, because "nothing" would be untrue.

- **To Anthropic:** everything Claude reads. Your resume, your claims register, postings, your interview answers. Governed by your own Claude plan's data terms, the same as any other conversation.
- **To public boards and feeds:** ordinary outbound requests, throttled to one per host every 0.7 seconds. CareerKit sends no profile identity fields to unauthenticated endpoints; normal network metadata such as your IP is visible.
- **To keyed feeds you enable:** configured API keys, account IDs or affiliate values go to that provider. USAJobs additionally requires your registered email. Skip any keyed feed whose disclosure you do not accept.
- **To Freehire, only if you enable it:** quoted search terms, optional country and age filters, and ordinary request metadata go to that third-party hosted discovery API. No API key, resume, claims register or application data is sent. CareerKit accepts only conservative first-party ATS source-and-host pairs and can, if separately enabled, hydrate thin results from the direct employer URL. Every Freehire row is forced to `VERIFY`; confirm it on the direct employer ATS, then ingest it for polling when CareerKit supports that ATS family. Unsupported families remain manual verification leads.
- **No telemetry.** Nothing is reported to the author or anyone else.

Delete `profile/`, `data/` and `out/` to remove CareerKit-owned local state. That cannot retract provider requests or erase backups, exports or copies held by other tools.

On scraping

Every feed declares what it is: an official public API, one that needs your own key, or a scraper. The optional LinkedIn guest feed reads public search pages and is off by default. The dormant JobSpy adapter has no supported installation until its dependency graph can use the security-fixed Markdown converter. Scrapers can be rate limited or blocked, and their terms of use are yours to read. Any run whose results include one says so in the report.

SEVEN

Failure modes worth knowing

A job-search engine that is subtly wrong does not crash. It hands you a clean, confident report with the wrong jobs in it. Most of the engineering here is aimed at that. Some of what it now defends against, all of it found in this code:

- Two different openings with the same title at one employer collapsing into one entry, so marking one applied silently hid the other.
- A blank entry in an exclusions list compiling to a pattern that matched every posting ever written. Zero results is indistinguishable from a quiet day.
- Postings that had closed continuing to surface as live, because nothing wrote back that they were gone.
- A stipend of $500 beside a real salary band being read as $500,000 and becoming the top of the range.
- "New since last time" meaning "first seen today", so two runs in one day announced the same roles twice.
- A consistency check whose pattern matched nothing, so it reported no problems forever. A check that cannot fail is worse than no check.

> THE TEST SUITE IS THE POINT
>
> The suite is written against defects that were really in the code rather than for coverage. Every case names a failure that had shipped, and several are parametrised over the exact inputs that broke. When a batch of fixes went in, the fixes themselves were attacked, and every serious defect in the second round had been introduced by a fix from the first. An exact count is deliberately not printed here: it was wrong twice while this guide was being written, which is what a hand-synced number does.

EIGHT

Reference

Layout

`profile/`	Yours. Profile, claims, style, employer registry, tracker. Gitignored.
`data/`	The SQLite database and HTTP cache. Gitignored.
`out/`	Generated reports, one per run, plus an immutable copy of each. Gitignored.
`engine/`	Adapters, feeds, scoring, storage, reporting. No personal data, ever.
`.claude/skills/`	The workflows Claude follows for each command.

Sharing it

`./new-instance.sh <name>` clones a fully separate instance with its own profile, database and reports. Instances never see each other's data. Use one per person. Instances clone from this checkout's HTTP(S) origin and print the exact URL; confirm the recipient can access it. Update one with `git pull && ./setup.sh` so dependency migrations arrive with engine fixes. Local or SSH-only origins require explicit `--local`.

Licence

MIT. Use it, fork it, change it. If you improve the engine, the improvement belongs in the template so every instance inherits it.

`github.com/omoji-personal/careerkit`

`Guide build b27bd82dd840`